

Node.js - Meri Full Revision Notes

Roman Urdu | Easy samajh + code + yaad rakhne wali baatein + practice

Note: Ye notes sirf un topics par based hain jo screenshots mein nazar aa rahe hain aur Modules ki woh practice jo tum kar chuke ho.

1. Node.js kya hai?

Node.js ek JavaScript runtime hai. Simple alfaaz mein: browser ke andar JavaScript chalti hai, lekin Node.js ki madad se hum JavaScript ko browser ke bahar bhi chala sakte hain. Isi wajah se JavaScript ko backend/server-side kaam ke liye bhi use kiya ja sakta hai.

```
console.log("Hello Node.js");  
  
// terminal  
node app.js
```

Yaad rakhna: Yaad rakho: Node.js koi nayi programming language nahi hai. Language JavaScript hi hai; Node.js us JavaScript ko chalane ka runtime/environment deta hai.

2. History aur Prerequisites

Is topic mein Node.js ka background aur start karne se pehle required knowledge aati hai. Tumhare current level par important cheez ye hai ke JavaScript basics clear hon: variables, functions, arrays aur objects.

Yaad rakhna: Quick check: agar tum arrow function, array aur object samajh lete ho to Modules ki practice zyada easy hoti hai.

3. Installation aur file run karna

Node.js install hone ke baad .js file terminal se run ki ja sakti hai. File project folder mein banao, terminal usi folder mein kholo aur node ke saath filename run karo.

```
node app.js
```

Yaad rakhna: Error aaye to sab se pehle terminal mein file path aur error wali line dekho.

4. REPL

REPL ek interactive jagah hai jahan chhota JavaScript expression type karte hi result mil jata hai. Chhoti testing ke liye useful hai; bari coding ke liye .js file better hoti hai.

```
node  
> 10 + 5  
15  
> 5 * 4  
20
```

Yaad rakhna: REPL = quick test. File = saved/reusable program.

5. Window vs Global

Browser aur Node.js ka environment same nahi hota. Browser web page ke liye window aur document jaisi cheezen deta hai. Node.js browser nahi hai; uska apna runtime/global environment hota hai.

```
Browser -> window, document  
Node.js -> global runtime environment
```

Yaad rakhna: Main concept: jis environment mein code chal raha hai, us environment ki available cheezen different ho sakti hain.

6. Module kya hai?

Module ka matlab code ko alag files mein divide karna. Example: math.js mein calculation function rakho aur app.js mein usko use karo. Is se code organized aur reusable hota hai.

```
// math.js
const add = (a,b) => a + b;
module.exports = add;

// app.js
const add = require("./math");
console.log(add(10,5));
```

Yaad rakhna: Flow yaad karo: function/value banao -> module.exports se bahar bhejo -> require() se doosri file mein lao -> use karo.

7. Single Export

Jab ek hi function ya value export karni ho to direct module.exports ke equal kar dete hain.

```
const greet = (name) => `Hello ${name}`;
module.exports = greet;

const greet = require("./greet");
console.log(greet("Junaid"));
```

Yaad rakhna: Single export formula: module.exports = functionName

8. Array Export

Multiple functions/values ko array [] mein rakh kar export kar sakte ho. Import ke baad array destructuring [] se values nikal sakte ho.

```
module.exports = [add, sub, multi, div];

const [add, sub, multi, div] = require("./math");
```

Yaad rakhna: Sab se important: [] = Array. Array destructuring mein order important hota hai.

9. Object Export

Multiple related functions ko object {} mein export karna bohat useful pattern hai. Har function object ki named property ban jata hai.

```
module.exports = { add, sub, multi, div };

const math = require("./math");
console.log(math.add(10,5));
```

Yaad rakhna: {} = Object. Pura object import karoge to math.add(), math.sub() jese access kar sakte ho.

10. Object Destructuring

Pura math object baar baar likhne ke bajaye object se functions direct nikal sakte ho.

```
const { add, sub, multi, div } = require("./math");

console.log(add(10,5));
console.log(sub(10,5));
```

Yaad rakhna: Object destructuring mein {} use hota hai aur property names match karte hain.

11. Tumhari ab tak ki practice

Tum single function export kar chuke ho. Tum array ke andar multiple functions export karke array destructuring kar chuke ho. Tum object ke andar add, sub, multi aur div export karke math.add() style se use bhi kar chuke ho. Is liye ab tumhara basic CommonJS module flow practical level par cover ho chuka hai.

```
Single -> module.exports = add
Array -> module.exports = [add, sub]
Object -> module.exports = { add, sub }

Import single -> const add = require('./math')
Import array -> const [add, sub] = require('./math')
Import object -> const { add, sub } = require('./math')
```

12. Solved Revision Task

Task: calculator.js mein add aur sub banao, object se export karo, phir app.js mein destructuring se import karo.

```
// calculator.js
const add = (a,b) => a+b;
const sub = (a,b) => a-b;
module.exports = { add, sub };

// app.js
const { add, sub } = require("./calculator");
console.log(add(20,10)); // 30
console.log(sub(20,10)); // 10
```

13. Khud Solve Karne Wale Tasks

1. greeting.js mein welcome(name) banao, single export karo aur app.js mein use karo.
2. subject.js mein 4 subject values/functions ko array se export karo aur app.js mein array destructuring karo.
3. math.js mein add, sub, multi, div aur remainder banao; object se export karo aur object destructuring se import karo.
4. Ek mini Student Utility System banao jisme single, array aur object tino export methods use hon.

14. 1-Minute Revision

Node.js = JavaScript ko browser ke bahar run karne ka runtime.

REPL = quick testing.

Module = separate reusable file.

module.exports = file se cheez bahar dena.

require() = doosri file ki exported cheez lana.

[] = Array.

{} = Object.

Array destructuring = `[ ]`.

Object destructuring = `{ }`.